SIAM J. COMPUT.
Vol. 22, No. 5, pp. 935–948, October 1993
© 1993 Society for Industrial and Applied Mathematics

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

SUFFIX ARRAYS: A NEW METHOD FOR ON-LINE STRING SEARCHES*

UDI MANBER†‡ AND GENE MYERS†§

Abstract. A new and conceptually simple data structure, called a suffix array, for on-line string searches is introduced in this paper. Constructing and querying suffix arrays is reduced to a sort and search paradigm that employs novel algorithms. The main advantage of suffix arrays over suffix trees is that, in practice, they use three to five times less space. From a complexity standpoint, suffix arrays permit on-line string searches of the type, "Is W a substring of A?" to be answered in time $O(P + \log N)$, where P is the length of W and N is the length of A, which is competitive with (and in some cases slightly better than) suffix trees. The only drawback is that in those instances where the underlying alphabet is finite and small, suffix trees can be constructed in $O(N)$ time in the worst case, versus $O(N \log N)$ time for suffix arrays. However, an augmented algorithm is given that, regardless of the alphabet size, constructs suffix arrays in $O(N)$ *expected* time, albeit with lesser space efficiency. It is believed that suffix arrays will prove to be better in practice than suffix trees for many applications.

Key words. string searching, string matching, pattern matching, suffix trees, algorithms, text indexing, inverted indices

AMS subject classifications. 68P10, 68P20, 68Q25, 68U15

1. Introduction.

Finding all instances of a string W in a large text A is an important pattern matching problem. There are many applications in which a fixed text is queried many times. In these cases, it is worthwhile to construct a data structure to allow fast queries. The *Suffix tree* is a data structure that admits efficient on-line string searches. A suffix tree for a text A of length N over an alphabet Σ can be built in $O(N \log |\Sigma|)$ time and $O(N)$ space [Wei73], [McC76]. Suffix trees permit on-line string searches of the type, "Is W a substring of A?" to be answered in $O(P \log |\Sigma|)$ time, where P is the length of W. We explicitly consider the dependence of the complexity of the algorithms on $|\Sigma|$, rather than assume that it is a fixed constant, because Σ can be quite large for many applications. Suffix trees can also be constructed in time $O(N)$ with $O(P)$ time for a query, but this requires $O(N|\Sigma|)$ space, which renders this method impractical in many applications.

Suffix trees have been studied and used extensively. A survey paper by Apostolico [Apo85] cites over forty references. Suffix trees have been refined from tries to minimum state finite automaton for the text and its reverse [BBE85], generalized to on-line construction [MR80], [BB86], real-time construction of some features is possible [Sli80], and suffix trees have been parallelized [AIL88]. Suffix trees have been applied to fundamental string problems such as finding the longest repeated substring [Wei73], finding all squares or repetitions in a string [AP83], computing substring statistics [AP85], approximate string matching [Mye86], [LV89], [CL90], and string comparison [EH88]. They have also been used to address other types of problems such as text compression [RPE81], compressing assembly code [FWM84], inverted indices [Car75], and analyzing genetic sequences [CHM86]. Galil [Gal85] lists a number of open problems concerning suffix trees and on-line string searching.

In this paper, we present a new data structure, called the *suffix array* [MM90], that is basically a sorted list of all the suffixes of A. When a suffix array is coupled with information about the *longest common prefixes* (*lcps*) of adjacent elements in the suffix array, string searches can be answered in $O(P + \log N)$ time with a simple augmentation to a classic binary search.

*Received by the editors May 5, 1989; accepted for publication (in revised form) June 3, 1992.

†Department of Computer Science, University of Arizona, Tucson, Arizona 85721.

‡Supported in part by National Science Foundation Presidential Young Investigator Award grant DCR-8451397, with matching funds from AT&T, and by National Science Foundation grant CCR-9002351.

§Supported in part by National Institute of Health grant R01 LM04960-01, and by National Science Foundation grant CCR-9002351.

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

The suffix array and associated *lcp* information occupy a mere $2N$ integers, and searches are shown to require at most $P + \lceil \log_2(N-1) \rceil$ single-symbol comparisons. To build a suffix array (but not its *lcp* information) one could simply apply any string-sorting algorithm such as the $O(N \log N)$ expected-time algorithm of Baer and Lin [BL89]. But such an approach fails to take advantage of the fact that we are sorting a collection of related suffixes. We present an algorithm for constructing a suffix array and its *lcp* information with $3N$ integers[1] and $O(N \log N)$ time *in the worst case*. Time could be saved by constructing a suffix tree first, and then building the array with a traversal of the tree [Ro82] and the *lcp* information with constant-time nearest ancestor queries [SV88] on the tree. But this will require more space. Moreover, the algorithms for direct construction are interesting in their own right.

Our approach distills the nature of a suffix tree to its barest essence: A sorted array coupled with another to accelerate the search. Suffix arrays may be used in lieu of suffix trees in many (but not all) applications of this ubiquitous structure. Our search and sort approach is distinctly different and, in theory, provides superior querying time at the expense of somewhat slower construction. Galil [Gal85, Prob. 9] poses the problem of designing algorithms that are not dependent on $|\Sigma|$ and our algorithms meet this criterion, i.e., $O(P + \log N)$ search time with an $O(N)$ space structure, independent of Σ. With a few additional and simple $O(N)$ data structures, we show that suffix arrays can be constructed in $O(N)$ expected time, also independent of Σ. This claim is true under the assumption that all strings of length N are equally likely and exploits the fact that for such strings, the expected length of the *longest* repeated substring is $O(\log N / \log |\Sigma|)$ [KGO83].

In practice, an implementation based on a blend of the ideas in this paper compares favorably with an implementation based on suffix trees. Our suffix array structure requires only $5N$ bytes on a VAX, which is three to five times more space efficient than any reasonable suffix tree encoding. Search times are competitive, but suffix arrays do require three to ten times longer to build. For these reasons, we believe that suffix arrays will become the data structure of choice for the many applications where the text is very large. In fact, we recently found that the basic concept of suffix arrays (sans the *lcp* and a provable efficient algorithm) has been used in the Oxford English Dictionary (OED) project at the University of Waterloo [GBS92]. Suffix arrays have also been used as a basis for a sublinear approximate matching algorithm [My93] and for performing all pairwise comparisons between sequences in a protein sequence database [BG90].

The paper is organized as follows. In §2, we present the search algorithm under the assumption that the suffix array and the *lcp* information have been computed. In §3, we show how to construct the sorted suffix array. In §4, we give the algorithm for computing the *lcp* information. In §5, we modify the algorithms to achieve better expected running times. We end with empirical results and comments about practice in §6.

2. Searching. Let $A = a_0 a_1 \cdots a_{N-1}$ be a large text of length N. Denote by $A_i = a_i a_{i+1} \cdots a_{N-1}$ the suffix of A that starts at position i. The basis of our data structure is a lexicographically sorted array, Pos, of the suffixes of A; namely, $Pos[k]$ is the start position of the kth smallest suffix in the set $\{A_0, A_1, \ldots A_{N-1}\}$. The sort that produces the array Pos is described in §3. For now, we assume that Pos is given; namely, $A_{Pos[0]} < A_{Pos[1]} < \ldots < A_{Pos[N-1]}$, where "$<$" denotes the lexicographical order.

For a string u, let u^p be the prefix consisting of the first p symbols of u if u contains more than p symbols, and u otherwise. We define the relation $<_p$ to be the lexicographical order of p-symbol prefixes; that is, $u <_p v$ if and only if $u^p < v^p$. We define the relations $\leq_p$, $=_p$,

[1] While the suffix array and *lcp* information occupy $2N$ integers, other N integers are needed during their construction. All the integers contain values in the range $[-N, N]$.

$\neq_p$, and $\geq_p$ in a similar way. Note that, for any choice of p, the Pos array is also ordered according to $\leq_p$, because $u < v$ implies $u \leq_p v$. All suffixes that have equal p-prefixes, for some $p < N$, must appear in consecutive positions in the Pos array, because the Pos array is sorted lexicographically. These facts are central to our search algorithm.

Suppose that we wish to find all instances of a string $W = w_0 w_1 \cdots w_{P-1}$ of length $P \leq N$ in A. Let $L_W = \min(k : W \leq_p A_{Pos[k]}$ *or* $k = N)$ and $R_W = \max(k : A_{Pos[k]} \leq_p W$ *or* $k = -1)$. Since Pos is in $\leq_p$-order, it follows that W matches $a_i a_{i+1} \cdots a_{i+P-1}$ if and only if $i = Pos[k]$ for some $k \in [L_W, R_W]$. Thus, if L_W and R_W can be found quickly, then the number of matches is $R_W - L_W + 1$ and their left endpoints are given by $Pos[L_W]$, $Pos[L_W + 1], \ldots Pos[R_W]$. But Pos is in $\leq_p$-order; hence, a simple binary search can find L_W and R_W using $O(\log N)$ comparisons of strings of size at most P; each such comparison requires $O(P)$ single-symbol comparisons. Thus, the Pos array allows us to find all instances of a string in A in time $O(P \log N)$. The algorithm is given in Fig. 1.

```
if W ≤_p A_Pos[0] then
    L_W ← 0
else if W >_p A_Pos[N−1] then
    L_W ← N
else
  {  (L, R) ← (0, N − 1)
     while R − L > 1 do
       {   M ← (L + R)/2
           if W ≤_p A_Pos[M] then
                R ← M
           else
                L ← M
       }
     L_W ← R
  }
```

FIG. 1. *An $O(P \log N)$ search for L_W.*

The algorithm in Fig. 1 is very simple, but its running time can be improved. We show next that the $\leq_p$-comparisons involved in the binary search need not be started from scratch in each iteration of the while loop. We can use information obtained from one comparison to speed up the ensuing comparisons. When this strategy is coupled with some additional precomputed information, the search is improved to $P + \lceil \log_2(N-1) \rceil$ single-symbol comparisons in the worst case, which is a substantial improvement.

Let $lcp(v, w)$ be the length of the longest common prefix of v and w. When we lexicographically compare v and w in a left-to-right scan that ends at the first unequal symbol, we obtain $lcp(v, w)$ as a by-product. We can modify the binary search in Fig. 1 by maintaining two variables, l and r, such that $l = lcp(A_{Pos[L]}, W)$, and $r = lcp(W, A_{Pos[R]})$. Initially, l is set by the comparison of W and $A_{Pos[0]}$ in line 1, and r is set in the comparison against $A_{Pos[N-1]}$ in line 3. Thereafter, each comparison of W against $A_{Pos[M]}$ in line 9 permits l or r to be appropriately updated in line 10 or 12, respectively. By so maintaining l and r, $h = \min(l, r)$ single-symbol comparisons can be saved when comparing $A_{Pos[M]}$ to W, because $A_{Pos[L]} =_l W =_r A_{Pos[R]}$ implies $A_{Pos[k]} =_h W$ for all k in $[L, R]$ including M. While this reduces the number of single-symbol comparisons needed to determine the $\leq_p$-order of a midpoint with respect to W, it turns out that the worst-case running time is still $O(P \log N)$ (e.g., searching $ac_{N-2}b$ for $c^{P-1}b$).

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

To reduce the number of single-symbol comparisons to $P + \lceil \log_2(N-1) \rceil$ in the worst case, we use precomputed information about the *lcp*s of $A_{Pos[M]}$ with each of $A_{Pos[L]}$ and $A_{Pos[R]}$. Consider the set of all triples (L, M, R) that can arise in the inner loop of the binary search of Fig. 1. There are exactly $N-2$ such triples, each with a unique midpoint $M \in [1, N-2]$, and for each triple $0 \le L < M < R \le N-1$. Suppose that (L_M, M, R_M) is the unique triple containing midpoint M. Let $Llcp$ be an array of size $N-2$ such that $Llcp[M] = lcp(A_{Pos[L_M]}, A_{Pos[M]})$ and let $Rlcp$ be another array of size $N-2$ such that $Rlcp[M] = lcp(A_{Pos[M]}, A_{Pos[R_M]})$. The construction of the two $(N-2)$-element arrays, $Llcp$ and $Rlcp$, can be interwoven with the sort producing Pos and will be shown in §4. For now, we assume that the $Llcp$ and $Rlcp$ arrays have been precomputed.

Consider an iteration of the search loop for triple (L, M, R). Let $h = \max(l, r)$ and let Δh be the difference between the value of h at the beginning and at the end of the iteration. Assuming, without loss of generality, that $r \le l = h$, there are three cases to consider,[2] based on whether $Llcp[M]$ is greater than, equal to, or less than h. The cases are illustrated in Fig. 2(a), 2(b), and 2(c), respectively. The vertical bars denote the *lcp*s between W and the suffixes in the Pos array (except for l and r, these *lcp*s are not known at the time we consider M). The shaded areas illustrate $Llcp[M]$. For each case, we must determine whether L_W is in the right half or the left half (the binary search step) and we must update the value of either l or r. It turns out that both these steps are easy to make:

Case 1 $Llcp[M] > l$ (Fig. 2(a)).
In this case, $A_{Pos[M]} =_{l+1} A_{Pos[L]} \neq_{l+1} W$, and so W must be in the right half and l is unchanged.

Case 2 $Llcp[M] = l$ (Fig. 2(b)).
In this case, we know that the first l symbols of $Pos[M]$ and W are equal; thus, we need to compare only the $l+1$st symbol, $l+2$nd symbol, and so on, until we find one, say, $l+j$, such that $W \neq_{l+j} Pos[M]$. The $l+j$th symbol determines whether L_W is in the right or left side. In either case, we also know the new value of r or l—it is $l+j$. Since $l = h$ at the beginning of the loop, this step takes $\Delta h + 1$ single-symbol comparisons.

Case 3 $Llcp[M] < l$ (Fig. 2(c)).
In this case, since W matched l symbols of L and $< l$ symbols of M, it is clear that L_W is in the left side and that the new value of r is $Llcp[M]$.

Hence, the use of the arrays $Llcp$ and $Rlcp$ (the $Rlcp$ array is used when $l < r$) reduces the number of single-symbol comparisons to no more than $\Delta h + 1$ for each iteration. Summing over all iterations and observing that $\Sigma \Delta h \le P$, the total number of single-symbol comparisons made in an on-line string search is at most $P + \lceil \log_2(N-1) \rceil$, and $O(P + \log N)$ time is taken in the worst-case. The precise search algorithm is given in Fig. 3.

3. Sorting. The sorting is done in $\lceil \log_2(N+1) \rceil$ stages. In the first stage, the suffixes are put in buckets according to their first symbol. Then, inductively, each stage further partitions the buckets by sorting according to twice the number of symbols. For simplicity of notation, we number the stages 1, 2, 4, 8, etc., to indicate the number of affected symbols. Thus, in the Hth stage, the suffixes are sorted according to the $\le_H$-order. For simplicity, we pad the suffixes by adding blank symbols, such that the lengths of all of them become $N+1$. This padding is not necessary, but it simplifies the discussion; the version of the algorithm detailed in Fig. 4 does not make this assumption. The first stage consists of a bucket sort according to the first symbol of each suffix. The result of this sort is stored in the Pos array and in another array BH of Boolean values which demarcates the partitioning of the suffixes into m_1 buckets

[2] The first two cases can be combined in the program. We use three cases only for description purposes.

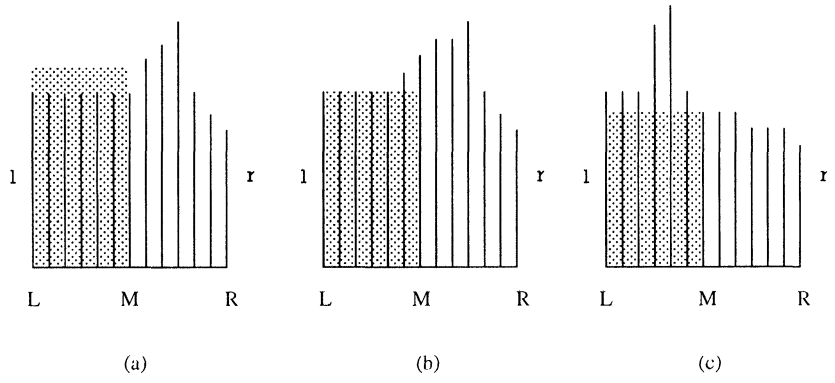


FIG. 2. *The three cases of the* $O(P + \log N)$ *search.*

```
l ← lcp(A_Pos[0], W)
r ← lcp(A_Pos[N−1], W)
if l = P or w_l ≤ a_Pos[0]+l then
        L_W ← 0
else if r < P or w_r ≤ a_Pos[N−1]+r then
        L_W ← N
else
  {     (L, R) ← (0, N − 1)
        while R − L > 1 do
          {     M ← (L + R)/2
                if l ≥ r then
                        if Lcp[M] ≥ l then
                                m ← l + lcp(A_Pos[M]+l, W_l)
                        else
                                m ← Lcp[M]
                else
                        ifRcp[M] ≥ r then
                                m ← r + lcp(A_Pos[M]+r, W_r)
                        else
                                m ← Rcp[M]
                if m = P or w_m ≤ a_Pos[M]+m then
                        (R, r) ← (M, m)
                else
                        (L, l) ← (M, m)
          }
        L_W ← R
  }
```

FIG. 3. *An* $O(P + \log N)$ *search for* L_W.

($m_1 \leq |\Sigma|$); each bucket holds the suffixes with the same first symbol. The array Pos will become progressively sorted as the algorithm proceeds. Assume that after the Hth stage the suffixes are partitioned into m_H buckets, each holding suffixes with the same H first symbols,

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

and that these buckets are sorted according to the $\leq_H$-relation. We will show how to sort the elements in each H-bucket to produce the $\leq_{2H}$-order in $O(N)$ time. Our sorting algorithm uses similar ideas to those in [KMR72].

Let A_i and A_j be two suffixes belonging to the same bucket after the Hth step; that is, $A_i =_H A_j$. We need to compare A_i and A_j according to the next H symbols. But, the next H symbols of A_i (A_j) are exactly the first H symbols of $A_{i+H}(A_{j+H})$. By the assumption, we already know the relative order, according to the $\leq_H$-relation, of A_{i+H} and A_{j+H}. It remains to see how we can use that knowledge to complete the stage efficiently. We first describe the main idea, and we then show how to implement it efficiently.

We start with the first bucket, which must contain the smallest suffixes according to the $\leq_H$-relation. Let A_i be the first suffix in the first bucket (i.e., $Pos[0] = i$), and consider A_{i-H} (if $i - H < 0$, then we ignore A_i and take the suffix of $Pos[1]$, and so on). Since A_i starts with the smallest H-symbol string, A_{i-H} should be the first in its $2H$-bucket. Thus, we move A_{i-H} to the beginning of its bucket and mark this fact. For every bucket, we need to know the number of suffixes in that bucket that have already been moved and thus placed in $\leq_{2H}$-order. The algorithm basically scans the suffixes as they appear in the $\leq_H$-order, and for each A_i it moves A_{i-H} (if it exists) to the next available place in its H-bucket. While this basic idea is simple, its efficient implementation (in terms of both space and time) is not trivial. We describe it below.

We maintain three integer arrays, Pos, Prm, and $Count$, and two Boolean arrays, BH and $B2H$, all with N elements.[3] At the start of stage H, $Pos[i]$ contains the start position of the ith smallest suffix (according to the first H symbols), $Prm[i]$ is the inverse of Pos, namely, $Prm[Pos[i]] = i$, and $BH[i]$ is 1 if and only if $Pos[i]$ contains the leftmost suffix of an H-bucket (i.e., $A_{Pos[i]} \neq_H A_{Pos[i-1]}$). $Count$ and $B2H$ are temporary arrays; their use will become apparent in the description of a stage of the sort. A radix sort on the first symbol of each suffix is easily tailored to produce Pos, Prm, and BH for stage 1 in $O(N)$ time. Assume that Pos, Prm, and BH have the correct values after stage H, and consider stage $2H$.

We first reset $Prm[i]$ to point to the leftmost cell of the H-bucket containing the ith suffix rather than to the suffix's precise place in the bucket. We also initialize $Count[i]$ to 0 for all i. All operations above can be done in $O(N)$ time. We then scan the Pos array in increasing order, one bucket at a time. Let l and r ($l \leq r$) mark the left and right boundary of the H-bucket currently being scanned. Let T_i (the H left extension of i) denote $Pos[i] - H$. For every i, $l \leq i \leq r$, we increment $Count[Prm[T_i]]$, set $Prm[T_i] = Prm[T_i] + Count[Prm[T_i]] - 1$, and set $B2H[Prm[T_i]]$ to 1. In effect, all the suffixes whose $H + 1$st through $2H$th symbols equal the unique H-prefix of the current H-bucket are moved to the top of their H-buckets with respect to the Prm array (Pos is updated momentarily). The $B2H$ field is used to mark

[3]We present a conceptually simpler but more space-expensive algorithm above in order to clearly expound the idea behind the sort. In fact, two N-element integer arrays are sufficient, and since the integers are always positive, we can use their sign bit for the Boolean values. Thus, the space requirement is only two integers per symbol. The trick is to remove the $Count$ array by temporarily using the Prm value of the leftmost suffix in a bucket to hold the count for the bucket. Instead of initializing $Count$ to 0 in the first step, we turn off the BH field of every bucket so that we can tell when a Prm value is the first reference to a particular bucket. The second step again consists of a scan of the Pos array. If $BH[Prm[T_i]]$ is off, then T_i is to be the first suffix in the $\leq_{2H}$ order of its H-bucket. We search the bucket for it and actually place it at the head of its bucket (as opposed to just modifying Prm to reflect where it will go in the simpler algorithm). This allows us to then use $Prm[T_i]$ as the counter for the bucket because we can restore it later knowing that the Prm-value of the first suffix in each bucket is being so used. We thus set the BH field back on and set $Prm[T_i]$ to 1. For later references to this bucket, which we know because $BH[Prm[T_i]]$ is now on, we simply adjust $Prm[T_i]$ with the count in $Prm[Pos[Prm[T_i]]]$ and bump the count. At the end of this step, the Prm fields used as counters are reset to the position of their suffix. The $B2H$ fields could not be set in the preceding steps because the BH values were being used as counter flags. In a separate pass, the $B2H$ values are updated to identify $2H$ buckets as in the simple algorithm.

those prefixes that were moved. Before the next H-bucket is considered, we make another pass through this one, find all the moved suffixes, and reset the $B2H$ fields such that only the leftmost of them in each $2H$-bucket is set to 1, and the rest are reset to 0. This way, the $B2H$ fields correctly mark the beginning of the $2H$-buckets. Thus the scan updates Prm and sets $B2H$ so that they are consistent with the $\leq_{2H}$-order of the suffixes. In the final step, we update the Pos array (which is the inverse of Prm) and set BH to $B2H$. All the steps above can clearly be done in $O(N)$ time, and, since there are at most $\lceil \log_2(N+1) \rceil$ stages, the sorting requires $O(N \log N)$ time in the worst case. A pseudocode implementation is given in Fig. 4. Average-case analysis is presented in §5.

```
# Sorting with 3N positive integers and
  2N Booleans. "Count" can be eliminated
  and Booleans folded into sign bits, to
  produce a 2N integer sort.      #

var Pos, Prm, Count: array [0..N − 1] of int;
    BH, B2H: array [0..N] of boolean;


# First phase bucket sort, Bucket and
  Link overlay Pos and Prm, respectively #

for a ∈ Σ do
    Bucket[a] ← −1
for i ← 0, 1, 2, ··· , N − 1 do
    (Bucket[a_i], Link[i]) ← (i, Bucket[a_i])
c ← 0
for a ∈ Σ in order do
  {  i ← Bucket[a]
     while i ≠ −1 do
       {  j ← Link[i]
          Prm[i] ← c
          if i = Bucket[a] then
            {  BH[c] ← true
               Set(c, 0)  # lcp info call #
            }
          else
               BH[c] ← false
          c ← c + 1
          i ← j
       }
  }
BH[N] ← true
for i ∈ [0, N − 1] do
    Pos[Prm[i]] ← i
```

```
# Inductive sorting stage (with lcp info calls) #

for H ← 1, 2, 4, 8, ··· while H < N do
  {  for each =_H bucket [l, r] do
       {  Count[l] ← 0
          for c ∈ [l, r] do
               Prm[Pos[c]] ← l
       }
     d ← N − H
     e ← Prm[d]
     Prm[d] ← e + Count[e]
     Count[e] ← Count[e] + 1
     B2H[Prm[d]] ← true
     for each =_H bucket [l, r] in ≤_H-order do
       {  for d ∈ {Pos[c] − H : c ∈ [l, r]} ∩ [0, N − 1] do
            {  e ← Prm[d]
               Prm[d] ← e + Count[e]
               Count[e] ← Count[e] + 1
               B2H[Prm[d]] ← true
            }
          for d ∈ {Pos[c] − H : c ∈ [l, r]} ∩ [0, N − 1] do
               if B2H[Prm[d]] then
                    {  e ← min(j : j > Prm[d] and
                                  (BH[j] or not B2H[j]))
                       for f ∈ [Prm[d] + 1, e − 1] do
                            B2H[f] ← false
                    }
       }
     for i ∈ [0, N − 1] do
         Pos[Prm[i]] ← i
     for i ∈ [0, N − 1] do
         if B2H[i] and not BH[i] then
           {  Set(i, H + Min_Height(Prm[Pos[i − 1] + H],
                                    Prm[Pos[i] + H]))
              BH[i] ← B2H[i]
           }
}
```

FIG. 4. *The $O(N \log N)$ suffix sorting algorithm.*

4. Finding longest common prefixes. The $O(P + \log N)$ search algorithm requires precomputed information about the *lcp*s between the suffixes starting at each midpoint M and its left and right boundaries L_M and R_M. Computing a suffix array requires $2N$ integers, and we will see here that computing and recording the associated *lcp* information requires an extra N integers. We first show how to compute the *lcp*s between suffixes that are *consecutive* in the

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

sorted Pos array during the sort. We will see later how to compute *all* necessary lcps. The key idea is the following. Assume that after stage H of the sort we know the lcps between suffixes in adjacent buckets (after the first stage, the lcps between suffixes in adjacent buckets are 0). At stage $2H$ the buckets are partitioned according to $2H$ symbols. Thus, the lcps between suffixes in newly adjacent buckets must be at least H and at most $2H-1$. Furthermore, if A_p and A_q are in the same H-bucket but are in distinct $2H$-buckets, then

$$lcp(A_p, A_q) = H + lcp(A_{p+H}, A_{q+H}). \tag{4.1}$$

Moreover, we know that $lcp(A_{p+H}, A_{q+H}) < H$. The problem is that we only have the lcps between suffixes in adjacent buckets, and A_{p+H} and A_{q+H} may not be in adjacent buckets. However, if $A_{Pos[i]}$ and $A_{Pos[j]}$, where $i < j$ have an lcp less than H and Pos is in $\leq_H$ order, then their lcp is the minimum of the lcps of every adjacent pair of suffixes between $Pos[i]$ and $Pos[j]$. That is,

$$lcp(A_{Pos[i]}, A_{Pos[j]}) = \min_{k\in[i,j-1]}(lcp(A_{Pos[k]}, A_{Pos[k+1]})). \tag{4.2}$$

Using (4.2) directly would require too much time, and maintaining the lcp of every pair of suffixes too much space. By using an $O(N)$-space height balanced tree structure that records the minimum pairwise lcp over a collection of intervals of the suffix array, we will be able to determine the lcp between any two suffixes in $O(\log N)$ time. We will describe this data structure, which we call an *interval tree*, after we firmly establish our basic approach (interval trees are similar to the Cartesian trees of Vuillemin [Vui80]).

We define $height(i) = lcp(A_{Pos[i-1]}, A_{Pos[i]})$, $1 \leq i \leq N-1$, where Pos is the final sorted order of the suffixes. These $N-1$ $height$ values are computed in an array $Hgt[i]$. The computation is performed inductively, together with the sort, such that $Hgt[i]$ achieves its correct value at stage H if and only if $height(i) < H$, and it is undefined (specifically, $N+1$) otherwise. Formally, if $height(i) < H$ then $Hgt[i] = height(i)$; otherwise, $Hgt[i] = N+1$. Notice that, if $height(i) < H$, then $A_{Pos[i-1]}$ and $A_{Pos[i]}$ must be in different H-buckets since H-buckets contain suffixes with the same H-symbol prefix. Further observe that a Hgt value is initially $N+1$, it is set to its $height$ value during the appropriate stage of the sort, and it retains this value thereafter.

Let Pos^H, Hgt^H, and Prm^H be the values of the given arrays at the end of stage H. In stage $2H$ of the sort, the $\leq_{2H}$-ordered list Pos^{2H} is produced by sorting the suffixes in each H-bucket of the $\leq_h$-ordered list Pos^H. The following lemma captures the essence of how we compute Hgt^{2H} from Hgt^H given Pos^{2H} and Prm^{2H}.

LEMMA 1. *If* $H \leq height(i) < 2H$ *then*

$$height(i) = H + \min(Hgt^H[k] : k \in [\min(a,b)+1, \max(a,b)]),$$

where $a = Prm^{2H}[Pos^{2H}[i-1]+H]$, *and* $b = Prm^{2H}[Pos^{2H}[i]+H]$.

Proof. Let $p = Pos^{2H}[i-1]$ and $q = Pos^{2H}[i]$. As we have observed, $height(i) < 2H$ implies $height(i) = H + lcp(A_{p+H}, A_{q+H})$. Next observe that $Pos^{2H}[a] = p+H$ and $Pos^{2H}[b] = q+H$ by the choice of a and b. Without loss of generality, assume that $a < b$. We now know that $height(i) = H + lcp(u, v)$, where $u = A_{Pos^{2H}[a]}$, $v = A_{Pos^{2H}[b]}$, $lcp(u,v) < H$, and $u <_H v$. Observe that $x <_H z$ and $x \leq_H y \leq_H z$ imply $lcp(x,z) = \min(lcp(x,y), lcp(y,z))$. It follows, by induction, that if $x_0 <_H x_n$ and $x_0 \leq_H x_1 \leq_H \cdots \leq_H x_n$, then $lcp(x_0, x_n) = \min(lcp(x_{k-1}, x_k) : k \in [1,n])$. Thus, $lcp(u,v) = \min(lcp(A_{Pos^{2H}[k-1]}, A_{Pos^{2H}[k]}) : k \in [a+1, b])$. Now $lcp(u,v) < H$ implies that at least one term in the minimum is less than H. For those terms less than H, $lcp(A_{Pos^{2H}[k-1]}, A_{Pos^{2H}[k]}) =$

$height(k) = Hgt^H[k]$. This, combined with the fact that $Hgt^H[k] = N + 1 \geq H$ for all other terms, gives the result. □

We are now ready to describe the algorithm. In the first stage, we set $Hgt[i]$ to 0 if $a_{Pos^1[i-1]} \neq a_{Pos^1[i]}$, and to $N + 1$ otherwise. This correctly establishes Hgt^1. At the end of stage $2H > 1$, we have computed Pos^{2H}, Prm^{2H}, and BH^{2H} (which marks the $2H$-buckets). Thus, by Lemma 1, the following code correctly establishes Hgt^{2H} from Hgt^H when placed at the end of a sorting stage. Essential to its correctness is the fact that Hgt^{2H} is Hgt^H except for the elements whose $height$ values are in the range $[H, 2H - 1]$; their Hgt values are changed from $N + 1$ to their correct value.

for $i \in [1, N - 1]$ such that $BH[i]$ and $Hgt[i] > N$ **do**
{ $a \leftarrow Prm[Pos[i - 1] + H]$
$b \leftarrow Prm[Pos[i] + H]$
$Set\ (i, H + Min_Height(\min(a, b) + 1, \max(a, b)))$ { these routines are defined below }
}

The routine $Set(i, h)$ sets $Hgt[i]$ from $N + 1$ to h in our interval tree, and $Min_Height(i, j)$ determines $\min(Hgt[k] : k \in [i, j])$ using the interval tree. We now show how to implement each routine in time $O(\log N)$ in the worst case. Consider a balanced and full binary tree with $N - 1$ leaves, which, in left-to-right order, correspond to the elements of the array Hgt. The tree has height $O(\log N)$ and $N - 2$ interior vertices. Assume that a value $Hgt[v]$ is also kept at each interior vertex v. We say that the tree is *current* if for every interior vertex v, $Hgt[v] = \min(Hgt[left(v)], Hgt[right(v)])$, where $left(v)$ and $right(v)$ are the left and right children of v.

Let T be a current tree. We need to perform two operations on the tree, a query $Min_Height(i, j)$, and a dynamic operation $Set(i, h)$. The query operation $Min_Height(i, j)$ computes $\min(Hgt[k] : k \in [i, j])$. It can be answered in $O(\log N)$ time as follows. Let $nca(i, j)$ be the nearest common ancestor of leaves i and j. The nca of leaves i and j can be found in $O(\log N)$ time by simply walking from the leaves to the root of the tree (it can be done in constant time using a more complicated data structure [SV88], but it is not necessary here). Let P be the set of vertices on the path from i to $nca(i, j)$ excluding $nca(i, j)$, and let Q be the similar path for leaf j. $Min_Height(i, j)$ is the minimum of the following values: (1) $Hgt[i]$, (2) $Hgt[w]$ such that $right(v) = w$ and $w \notin P$ for some $v \in P$, (3) $Hgt[w]$ such that $left(v) = w$ and $w \notin Q$ for some $v \in Q$, and (4) $Hgt[j]$. These $O(\log N)$ vertices can be found, and their minimum can be computed in $O(\log N)$ time. The operation $Set(i, h)$ sets $Hgt[i]$ to h and then makes T current again by updating the Hgt values of the interior vertices on the path from i to the root. This takes $O(\log N)$ time.

Overall, the time taken to compute the $height$ values in stage H is $O(N + \log N \cdot Set_H)$, where Set_H is the number of indices i for which $height(i) \in [H, 2H - 1]$. Since $\Sigma\, Set_H = N$ over all stages, the total additional time required to compute Hgt during the sort is $O(N \log N)$.

The Hgt array gives the $lcps$ of suffixes that are consecutive in the Pos array. Moreover, an interior vertex of our interval tree gives the lcp between the suffixes at its leftmost and rightmost leaves. We now show that not only are the arrays $Llcp$ and $Rlcp$ computable from the array Hgt but are directly available from the interval tree by appropriately choosing the shape of the tree (heretofore we only asserted that it needed to be full and balanced). Specifically, we use the tree based on the binary search of Fig. 1. This implicitly-represented tree consists of $2N - 3$ vertices, each labeled with one of the $2N - 3$ pairs (L, R) that can arise at entry and exit from the while loop of the binary search. The root of the tree is labeled $(0, N - 1)$, and the remaining vertices are labeled either (L_M, M) or (M, R_M) for some midpoint $M \in [1, N - 2]$. From another perspective, the tree's $N - 2$ interior

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

vertices are (L_M, R_M) for each midpoint M, and its $N-1$ leaves are $(i-1, i)$ for $i \in [1, N-1]$ in left-to-right order. For each interior vertex, $left((L_M, R_M)) = (L_M, M)$ and $right((L_M, R_M)) = (M, R_M)$. Since the tree is full and balanced, it is appropriate for realizing *Set* and *Min_Height* if we let leaf $(i-1, i)$ hold the value of $Hgt[i]$. Moreover, at the end of the sort, $Hgt[(L, R)] = \min(height(k) : k \in [L+1, R]) = lcp(A_{Pos[L]}, A_{Pos[R]})$. Thus, $Llcp[M] = Hgt[(L_M, M)]$ and $Rlcp[M] = Hgt[(M, R_M)]$. So with this tree, the arrays *Llcp* and *Rlcp* are directly available upon completion of the sort.[4]

5. Linear time expected-case variations. We now consider the expected time complexity of constructing and searching suffix arrays. The variations presented in this section require additional $O(N)$ structures and so lose some of the space advantage. We present them primarily to show that linear time constructions are possible and because some of the ideas here are central to the implementation we found to be best in practice. We assume that all N-symbol strings are equally likely.[5] Under this input distribution, the expected length of the longest repeated substring has been shown to be $2\log_{|\Sigma|} N + O(1)$ [KGO83]. This fact provides the central leverage for all the results that follow. Note that it immediately implies that, in the expected case, *Pos* will be completely sorted after $O(\log\log N)$ stages, and the sorting algorithm of §3 thus takes $O(N \log\log N)$ expected time.

The expected sorting time can be reduced to $O(N)$ by modifying the radix sort of the first stage as follows. Let $T = \lfloor \log_{|\Sigma|} N \rfloor$ and consider mapping each string of T symbols over Σ to the integer obtained when the string is viewed as a T-digit, radix-$|\Sigma|$ number. This frequently used encoding is an isomorphism onto the range $[0, |\Sigma|^T - 1] \subseteq [0, N-1]$, and the $\leq$-relation on the integers is identical with the $\leq_T$-relation on the corresponding strings. Let $Int_T(A_p)$ be the integer encoding of the T-symbol prefix of suffix A_p. It is easy to compute $Int_T(A_p)$ for all p in a single $O(N)$ sweep of the text by employing the observation that $Int_T(A_p) = a_p|\Sigma|^{T-1} + \lfloor Int_T(A_{p+1})/|\Sigma| \rfloor$. Instead of performing the initial radix sort on the first symbol of each suffix, we perform it on the integer encoding of the first T symbols of each suffix. This radix sort still takes just $O(N)$ time and space because the choice of T guarantees that the integer encodings are all less than N. Moreover, it sorts the suffixes according to the $\leq_T$-relation. Effectively, the base case of the sort has been extended from $H = 1$ to $H = T$ with no loss of asymptotic efficiency. Since the expected length of the longest repeated substring is $T \cdot (2 + O(1/T))$, at most, two subsequent stages are needed to complete the sort in the expected case. Thus, this slight variation gives an $O(N)$ expected algorithm for sorting the suffixes.

Corresponding expected-case improvements for computing the *lcp* information, in addition to the sorted suffix array, are harder to come by. We can still achieve $O(N)$ expected-case time as we now show. We employ an approach to computing $height(i)$ that uses identity (4.1) recursively to obtain the desired *lcp*s. Let the *sort history* of a particular sort be the tree that models the successive refinement of buckets during the sort. There is a vertex for each H-bucket except those H-buckets that are identical to the $(H/2)$-buckets containing them. The sort history thus has $O(N)$ vertices, as each leaf corresponds to a suffix and each interior

[4]The interval tree requires $2N-3$ positive integers. However, the observation that one child of each interior vertex has the same value as its parent, permits interval trees (and thus the *Llcp* and *Rlcp* arrays) to be encoded and manipulated as $N-1$ signed integers. Specifically, if the number at a vertex is positive, then the vertex contains the *Llcp* values; if it is negative, then its positive part is the *Rlcp* value. The other value is that of its parent. One must store both the *Llcp* and *Rlcp* values for the root but this is only one extra integer. Note that in order to have both the *Llcp* and *Rlcp* values available at a vertex we must descend to it from the root. This is naturally the case for searches, and for *Set* and *Min_height* we simply traverse from root to leaf and back again at no increase in asymptotic complexity.

[5]The ensuing results also hold under the more general model where each text is assumed to be the result of N independent Bernoulli trials of a $|\Sigma|$-sided coin toss that is not necessarily uniform.

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

vertex has at least two children. Each vertex contains a pointer to its parent, and each interior vertex also contains the stage number at which its bucket was split. The leaves of the tree are assumed to be arranged in an N element array, so that the singleton bucket for suffix A_p can be indexed by p. It is a straightforward exercise to build the sort history in $O(N)$ time overhead during the sort. Notice that we determine the values $height(i)$ only after the sort is finished.

Given the sort history produced by the sort, we determine the lcp of A_p and A_q as follows. First, we find the nearest common ancestor (nca) of suffixes A_p and A_q in the sort history using an $O(1)$ time nca algorithm [HT84], [SV88]. The stage number H associated with this ancestor tells us that $lcp(A_p, A_q) = H + lcp(A_{p+H}, A_{q+H}) \in [H, 2H - 1]$. We then recursively find the lcp of A_{p+H} and A_{q+H} by finding the nca of suffixes A_{p+H} and A_{q+H} in the history, and so on, until an nca is discovered to be the root of the history. At each successive level of the recursion, the stage number of the nca is at least halved, and so the number of levels performed is $O(\log L)$, where L is the lcp of A_p and A_q. Because the longest repeated substring has expected length $O(\log_{|\Sigma|} N)$, the $N - 1$ lcp values of adjacent sorted suffices are found in $O(N \log\log N)$ expected time.

The scheme above can be improved to $O(N)$ expected time by strengthening the induction basis as was done for the sort. Suppose that we stop the recursion above when the stage number of an nca becomes less than $T' = \lfloor \frac{1}{2} \log_{|\Sigma|} N \rfloor$. Our knowledge of the expected maximum lcp length implies that, on average, only three or four levels are performed before this condition is met. Each level takes $O(1)$ time, and we are left having to determine the lcp of two suffices (say, A_p and A_q) that is known to be less than T'. To answer this final lcp query in constant time, we build a $|\Sigma|^{T'}$-by-$|\Sigma|^{T'}$ array $Lookup$, where $Lookup[Int_{T'}(x), Int_{T'}(y)] = lcp(x, y)$ for all T'-symbol strings x and y. By the choice of T' there are no more than N entries in the array, and they can be computed incrementally in an $O(N)$ preprocessing step along with the integer encodings $Int_{T'}(A_p)$ for all p. So for the final level of the recursion, $lcp(A_p, A_q) = Lookup[Int_{T'}(A_p), Int_{T'}(A_q)]$ may be computed in $O(1)$ time via table lookup. In summary, we can compute the lcp between any two suffixes in $O(1)$ expected time, and so can product the lcp array in $O(N)$ expected time.

The technique of using integer encodings of $O(\log N)$-symbol strings to speedup the expected preprocessing times, also provides a *pragmatic* speed-up for searching. For any $K \leq T$, let $Buck[k] = \min\{i : Int_K(A_{Pos[k]}) = i\}$. This bucket array contains $|\Sigma|^K$ nondecreasing entries and can be computed from the ordered suffix array in $O(N)$ additional time. Given a word W, we know immediately that L_W and R_W are between $Buck[k]$ and $Buck[k+1]-1$ for $k = Int_K(W)$. Thus, in $O(K)$ time we can limit the interval to which we apply the search algorithm proper to one whose average size is $N/|\Sigma|^K$. Choosing K to be T or very close to T, implies that the search proper is applied to an $O(1)$ expected-size interval and thus consumes $O(P)$ time in expectation *regardless* of whether the algorithm of Fig. 1 or Fig. 3 is used. While the use of bucketing does not asymptotically improve either worst-case or expected-case times, we found this speed-up very important in practice.

6. Practice. A primary motivation for this paper was to be able to efficiently answer on-line string queries for very long genetic sequences (on the order of one million or more symbols long). In practice, it is the space overhead of the query data structure that limits the largest text that may be handled. Throughout this section, we measure space in numbers of integers where typical current architectures model each integer in 4 bytes. Suffix trees are quite space expensive, requiring roughly 4 integers of overhead per text character. Utilizing an appropriate blend of the suffix array algorithms given in this paper, we developed an implementation requiring 1.25 integers of overhead per text character whose construction and search speeds are competitive with suffix trees.

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

There are three distinct ways to implement a data structure for suffix trees, depending on how the outedges of an interior vertex are encoded. We characterize the space occupied by that part of the structure needed for searches and ignore the extra integer (for "suffix links") needed during the suffix tree's construction. Using a $|\Sigma|$-element vector to model the outedges gives a structure requiring $2N + (|\Sigma| + 2) \cdot I$ integers, where I is the number of interior nodes in the suffix tree. Encoding each set of outedges with a binary search tree requires $2N + 5I$ integers. Finally, encoding each outedge set as a linked list requires $2N + 4I$ integers. The parameter $I < N$ varies as a function of the text. The first four columns of Table 1 illustrate the value of I/N and the per-text-symbol space consumption of each of the three coding schemes assuming that an integer occupies 4 bytes. These results suggest that the linked scheme is the most space parsimonious. We developed a tightly coded implementation of this scheme for the timing comparisons with our suffix array software.

TABLE 1
Empirical results for texts of length 100,000.

	Space (Bytes/text symbol)					**Construction Time**		**Search Time**	
		S. Trees			*S. Arrays*	*S. Trees*	*S. Arrays*	*S. Trees*	*S. Arrays*
	I/N	Link	Tree	Vector					
Random ($\|\Sigma\| = 2$)	.99	23.8	27.8	19.8	5.0	2.6	7.1	6.0	5.8
Random ($\|\Sigma\| = 4$)	.62	17.9	20.4	18.9	5.0	3.1	11.7	5.2	5.6
Random ($\|\Sigma\| = 8$)	.45	15.2	17.0	20.8	5.0	4.6	11.4	5.8	6.6
Random ($\|\Sigma\| = 16$)	.37	13.9	15.4	30.6	5.0	6.9	11.6	9.2	6.8
Random ($\|\Sigma\| = 32$)	.31	13.0	14.2	46.2	5.0	10.9	11.7	10.2	7.0
Text ($\|\Sigma\| = 96$)	.54	16.6	18.8	220.0	5.0	5.3	28.3	22.4	9.5
Code ($\|\Sigma\| = 96$)	.63	18.1	20.6	255.0	5.0	4.2	35.9	29.3	9.0
DNA ($\|\Sigma\| = 4$)	.72	19.5	22.4	25.2	5.0	2.9	18.7	14.6	9.2

For our practical implementation, we chose to build just a suffix array and use the radix-N initial bucket sort described in §5 to build it in $O(N)$ expected time. Without the *lcp* array the search must take $O(P \log N)$ worst-case time. However, keeping variables l and r as suggested in arriving at the $O(P + \log N)$ search significantly improves search speed in practice. We further accelerate the search to $O(P)$ expected time by using a bucket table with $K = \log_{|\Sigma|} N/4$ as described in §5. Our search structure thus consists of an N integer suffix array and an $N/4$ integer bucket array, and so consumes only 1.25 integers/5 bytes per text symbol assuming an integer is 4 bytes. As discussed in §3, $2N$ integers are required to construct the suffix array (without *lcp* information). So constructing an array requires a little more space than is required by the search structure, as is true for suffix trees. Given that construction is usually once only, we choose to compare the sizes of the search structures in Table 1.

Table 1 summarizes a number of timing experiments on texts of length 100,000. All times are in seconds and were obtained on a VAX 8650 running UNIX. Columns 6 and 7 give the times for constructing the suffix tree and suffix array, respectively. Columns 8 and 9 give the time to perform 100,000 successful queries of length 20 for the suffix tree and array, respectively. In synopsis, suffix arrays are 3–10 times more expensive to build, 2.5–5 times more space efficient, and can be queried at speeds comparable to suffix trees.

Acknowledgment. The authors wish to thank the referees for the insightful comments, especially one anonymous referee whose meticulous comments were beyond the call of duty.

REFERENCES

[Apo85] A. APOSTOLICO, *The myriad virtues of subword trees*, in Combinatorial Algorithms on Words, A. Apostolico and Z. Galil, eds., NATO ASI Series F: Computer and System Sciences, Vol. 12, Springer-Verlag, Berlin, New York, 1985, pp. 85–96.

[AIL88] A. APOSTOLICO, C. ILIOPOULOS, G. LANDAU, B. SCHIEBER, AND U. VISHKIN, *Parallel construction of a suffix tree with applications*, Algorithmica, 3 (1988), pp. 347–366.

[AP83] A. APOSTOLICO AND F. P. PREPARATA, *Optimal off-line detection of repetitions in a string*, Theoret. Comput. Sci., 22 (1983), pp. 297–315.

[AP85] ———, *Structural properties of the string statistics problem*, J. Comput. System Sci., 31 (1985), pp. 394–411.

[BL89] J. BAER AND Y. LIN, *Improving quicksort performance with a codeword data structure*, IEEE Trans. Software Engrg., 15 (1989), pp. 622–631.

[BG90] R. BAEZA-YATES AND G. GONNET, *All-against-all sequence matching*, Tech. Rep., Dept. of Computer Science, University of Chile, 1990.

[BB86] J. BLUMER AND A. BLUMER, *On-line Construction of a Complete Inverted File*, Tech. Rep. UCSC-CRL-86-11, Department of Computer Science, University of Colorado, Boulder, CO, 1986.

[BBE85] J. BLUMER, A. BLUMER, E. EHRENFEUCHT, D. HAUSSLER, M. T. CHEN, AND J. SEIFERAS, *The smallest automation recognizing the subwords of a text*, Theoret. Comput. Sci., 40 (1985), pp. 31–35.

[Car75] A. F. CARDENAS, *Analysis and performance of inverted data base structures*, Comm. ACM, 18 (1975), pp. 253–263.

[CHM86] B. CLIFT, D. HAUSSLER, R. MCCONNELL, T. D. SCHNEIDER, AND G. D. STORMO, *Sequence landscapes*, Nucleic Acids Research, 4 (1986), pp. 141–158.

[CL90] W. I. CHANG AND E. L. LAWLER, *Approximate string matching in sublinear expected time*, Proc. IEEE 31st Annual Symp. on Foundations of Computer Science, St. Louis, MO, October 1990, pp. 116–124.

[EH88] A. EHRENFEUCHT AND D. HAUSSLER, *A new distance metric on strings computable in linear time*, Discrete Appl. Math., 40 (1988), pp. 191–203.

[FWM84] C. FRASER, A. WENDT, AND E. W. MYERS, *Analyzing and compressing assembly code*, Proceedings of the SIGPLAN Symposium on Compiler Construction, 1984, pp. 117–121.

[Gal85] Z. GALIL, *Open problems in stringology*, in Combinatorial Algorithms on Words, A. Apostolico and Z. Galil., eds., NATO ASI Series F: Computer and System Sciences, Vol. 12, Springer-Verlag, Berlin, New York, 1985, pp. 1–8.

[GBS92] G. GONNET, R. BAEZA-YATES, AND T. SNIDER, *New indices for text: PAT trees and PAT arrays*, in Information Retrieval: Data Structures and Algorithms, B. Frakes and R. Baeza-Yates, eds., Prentice-Hall, Englewood Cliffs, NJ, 1992.

[HT84] D. HAREL AND R. E. TARJAN, *Fast algorithms for finding nearest common ancestors*, SIAM J. Comput., 13 (1984), pp. 338–355.

[KGO83] S. KARLIN, G. GHANDOUR, F. OST, S. TAVARE, AND L. J. KORN, *New approaches for computer analysis of nucleic acid sequences*, Proc. Natl. Acad. Sci. USA, 80 (September 1983), pp. 5660–5664.

[KMR72] R. M. KARP, R. E. MILLER, AND A. L. ROSENBERG, *Rapid identification of repeated patterns in strings, trees and arrays*, Fourth Annual ACM Symposium on Theory of Computing, (May 1972), pp. 125–136.

[LV89] G. M. LANDAU AND U. VISHKIN, *Fast parallel and serial approximate string matching*, J. Algor., 10 (1989), pp. 157–169.

[McC76] E. M. MCCREIGHT, *A space-economical suffix tree construction algorithm*, J. ACM, 23 (1976), pp. 262–272.

[MM90] U. MANBER AND E. W. MYERS, *Suffix arrays: A new method for on-line string searches*, First ACM-SIAM Symposium on Discrete Algor., San Francisco, CA, January 1990, pp. 319–327.

[MR80] M. E. MAJSTER AND A. REISER, *Efficient on-line construction and correction of position trees*, SIAM J. Comput., 9 (1980), pp. 785–807.

[Mye86] E. W. MYERS, *Incremental alignment algorithms and their applications*, Tech. Report TR86-22, Department of Computer Science, University of Arizona, Tucson, AZ.

[My93] ———, *A sublinear algorithm for approximate keyword searching*, Algorithmica, 1993, to appear.

[Ro82] M. RODEH, *A fast test for unique decipherability based on suffix tree*, IEEE Trans. Inform. Theory, 28 (1982), pp. 648–651.

[RPE81] M. RODEH, V. R. PRATT, AND S. EVEN, *Linear algorithm for data compression via string matching*, J. ACM, 28 (1982), pp. 16–24.

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

[Sli80] A. O. SLISENKO, *Detection of periodicities and string-matching in real time*, J. Soviet Math., 22 (1983), pp. 1316–1387; translated from Zpiski Nauchnykh Seminarov Leningradskogo Otdeleniya Matematicheskogo Instituta im. V. A. Steklova AN SSSR, 105 (1980), pp. 62–173.

[SV88] B. SCHIEBER AND U. VISHKIN, *On finding lowest common ancestors: Simplification and parallelization*, SIAM J. Comput., 17 (1988), pp. 1253–1262.

[Vui80] J. VUILLEMIN, *A unified look at data structures*, Comm. ACM, 23 (1980), pp. 229–239.

[Wei73] P. WEINER, *Linear pattern matching algorithm*, Proc. 14th IEEE Sympos. Switching and Automata Theory, (1973), pp. 1–11.

Downloaded 11/10/26 to 128.220.159.217 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy